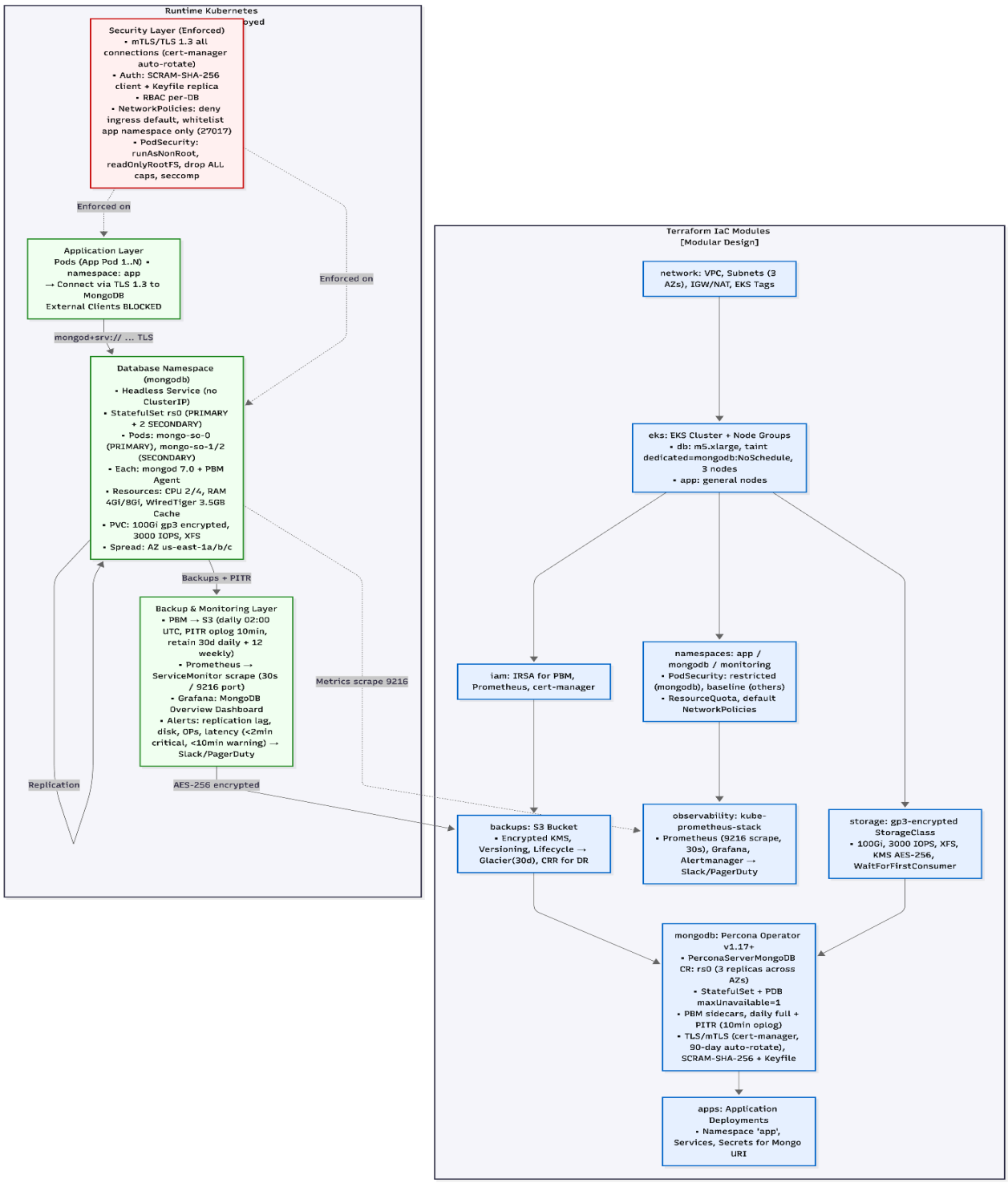
IAC to set up this infrastructure



1. Repository Layout

```
terraform/
  envs/
    prod/
      main.tf # Compose modules for prod
      providers.tf
      backend.tf
      variables.tf
      outputs.tf
  modules/
    network/ # VPC, subnets, routing
    eks/ # EKS cluster and node groups
    iam/ # IRSA roles and IAM policies
    storage/ # StorageClasses, CSI configuration
    namespaces/ # Namespaces + baseline security
    observability/ # Prometheus, Grafana, Alertmanager
    backups/ # S3 bucket + KMS, lifecycle
    mongodb/ # operator, replSet, PBM
    apps/ # App deployments, services, NetworkPolicies
```

---

2. Root (Environment) Configuration – envs

providers.tf

Configuration needed:

- aws provider for the chosen region
- kubernetes provider bound to the EKS cluster (endpoint + CA + token)
- helm provider also bound to the EKS cluster

The EKS cluster outputs (endpoint, CA) from the eks module feed into kubernetes and helm providers. We use aws_eks_cluster_auth data source to get a token for authentication.

main.tf

This is where I compose all the lower-level modules:

- network: creates VPC + subnets
- eks: creates EKS cluster and node groups using private subnets
- iam: creates IRSA roles for PBM, Prometheus, etc.
- storage: creates StorageClasses using AWS EBS CSI
- namespaces: creates app, mongodb, monitoring namespaces and PodSecurity labels
- observability: deploys kube-prometheus-stack (Prometheus, Grafana, Alertmanager)
- backups: creates S3 bucket with encryption and versioning for MongoDB backups
- mongodb: deploys Percona operator, MongoDB replica set, PBM backup configuration, TLS
- apps: deploys application workloads, services, and network policies

This layer only glues modules via input variables and outputs. All per-environment differences are passed as variables.

---

3. Network Module – modules/network

- Creates a dedicated VPC with:
    - CIDR (e.g., 10.0.0.0/16)
    - Public and private subnets across at least three AZs
    - Internet gateway, NAT gateways, and route tables
- Tags subnets for EKS and load balancers:
    - kubernetes.io/cluster/ = shared
    - kubernetes.io/role/internal-elb = 1 for private subnets
    - kubernetes.io/role/elb = 1 for public subnets

Inputs

- vpc_cidr
- azs
- env

Outputs

- vpc_id
- private_subnet_ids
- public_subnet_ids

---

4. EKS Module – modules/eks

- Creates EKS control plane and managed node groups:
    - eks_managed_node_group for “db” workloads
    - eks_managed_node_group for “app” workloads
- Enables IRSA (IAM Roles for Service Accounts)

db node group

- Instance type tuned for storage and memory (e.g., m6i.large or r6i.large - need to check pricing)
- Desired size: 3 nodes (one per AZ)
- Labels: role = db
- Taints: role=db:NoSchedule

app node group

- Instance type tuned for app workloads (probably m6i.large or similar)
- No taints; labeled role = app

Inputs

- cluster_name, cluster_version
- vpc_id, private_subnet_ids
- db_node_instance_type, app_node_instance_type

Outputs

- cluster_name
- cluster_endpoint
- cluster_ca_certificate
- oidc_provider_arn

---

5. IAM Module – modules/iam

- Creates IAM roles and policies bound to Kubernetes service accounts via IRSA for:
    - PBM (Percona Backup for MongoDB)
    - Prometheus or other monitoring components
    - Optional controllers
- Use aws_iam_role with assume_role_policy referencing the EKS OIDC provider
- Restrict conditions with the service account subject
- Apply least privilege S3 access

Inputs

- cluster_oidc_provider (ARN)
- oidc_provider_url
- backup_bucket

Outputs

- pbm_irsa_role_arn
- prometheus_irsa_role_arn

---

6. Storage Module – modules/storage

- Defines StorageClasses using AWS EBS CSI driver tuned for MongoDB
- Encryption via KMS
- Performance tuning
- Retain reclaim policy
- WaitForFirstConsumer binding

Inputs

- kms_key_id
- performance parameters

Outputs

- db_storage_class

---

7. Namespaces & Baseline Security – modules/namespaces

- Creates namespaces: app, mongodb, monitoring
- Applies Pod Security labels
- Optional default NetworkPolicies

---

8. Observability Module – modules/observability

- Deploys kube-prometheus-stack
- Alertmanager + Grafana configuration
- ServiceMonitor resources

Inputs

- slack_webhook
- iam_irsa_arn

---

9. Backups Module – modules/backups

- Creates encrypted S3 backup bucket
- Versioning + lifecycle rules
- Optional cross-region replication

Inputs

- env
- kms_key_id
- replication configuration

Outputs

- backup_bucket_name

---

10. MongoDB Module – modules/mongodb

- Deploys Percona MongoDB operator and replica set
- Storage + backup integration
- TLS/mTLS + SCRAM authentication

Key configuration highlights

- 3-node replica set
- AZ-aware scheduling
- PBM backups with PITR
- TLS enforcement

---

11. Application Module – modules/apps

- Deploys application workloads
- Configures MongoDB connectivity
- Applies network and pod security

Key patterns:

- Headless service DNS usage
- Secret-based credentials
- Node scheduling constraints

---

12. Step-by-Step Provisioning Order

1. Apply network module
2. Apply eks module
3. Apply iam module
4. Apply storage module
5. Apply namespaces module
6. Install cluster add-ons
7. Apply observability module
8. Apply backups module
9. Apply mongodb module
10. Apply apps module